FUNDAMENTOS DE TERRAFORM · CLASE 2

Resumen: Arquitectura interna y primer proyecto

Instructor: Ing. Álvaro Socop

1. Tres problemas reales que Terraform soluciona

En la Clase 1 vimos qué es IaC. Hoy vemos **por qué las empresas lo necesitan**: estos tres problemas aparecen en casi todos los equipos que administran infraestructura a mano.

Problema	Qué pasa sin IaC	Cómo lo resuelve Terraform
Snowflake servers	Cada servidor es único, con parches aplicados a mano y sin documentar. Si el servidor muere, el conocimiento muere con él.	El servidor **es código** en el repositorio: cualquiera lo recrea en minutos y los cambios pasan por pull request.
Configuration drift	La realidad se aleja del código con el tiempo: alguien abre el puerto 22 en la consola, otro aplica un parche… hasta que hay un incidente y nadie sabe por qué.	`terraform plan` compara el código con la realidad y **muestra la diferencia** antes de que cause problemas.
Infra no reproducible	"Funciona en prod, pero en QA está roto". Preparar el ambiente de un dev nuevo toma días y es imposible auditar quién cambió qué.	Un mismo código crea **N ambientes idénticos**. `git clone` + `terraform apply` y listo. El historial de Git es la auditoría.

> **Para recordar**
>
> "Un snowflake server es como una receta que solo el chef conoce. Si el chef se va, pierdes el plato."

2. Terraform en la industria

Terraform usa **el mismo lenguaje (HCL) y el mismo flujo** sin importar la nube. Lo único que cambia es el **provider**: el plugin que sabe hablar con la API de cada proveedor. En el Terraform Registry hay más de 3,000.

Nube	Provider	Para qué destaca
AWS	`hashicorp/aws`	El más usado del mundo. Más de 1,000 recursos: EC2, S3, RDS, VPC, IAM…
Azure	`hashicorp/azurerm`	VMs, AKS, SQL, Blob Storage. Muy demandado en empresas; se integra con Azure DevOps.
GCP	`hashicorp/google`	GKE, BigQuery, Cloud Run. Fuerte en cargas de ML/AI con Vertex AI.

```
# AWS
provider "aws" { region = "us-east-1" }

# Azure: mismo lenguaje
provider "azurerm" { features {} }

# GCP
provider "google" {
  project = "mi-proyecto"
  region  = "us-central1"
}
```

3. Instalación

Sistema	Cómo instalar
Windows	`winget install Hashicorp.Terraform` o, con Chocolatey, `choco install terraform`
macOS	`brew tap hashicorp/tap` y luego `brew install hashicorp/tap/terraform`
Linux / WSL	Agregar el repositorio oficial de HashiCorp con apt y ejecutar `sudo apt-get install terraform`
Verificar	`terraform version`

tfenv permite tener varias versiones de Terraform y usar la correcta en cada proyecto (como `nvm` en Node.js). La versión se fija en un archivo `.terraform-version`, algo esencial en pipelines de CI/CD. Comandos: `tfenv install 1.7.5`, `tfenv use 1.7.5`, `tfenv list`. Funciona en macOS, Linux y WSL.

4. ¿Cómo piensa Terraform internamente?

Archivos .tf (tu código) → **Terraform Core** (arma el grafo de dependencias) → **Provider plugin** (hoy: `hashicorp/local`) → **API o sistema real** (AWS, Azure… o el disco de tu computadora).

Concepto	Qué es
Desired State	El "deber ser": lo que define tu código .tf.
Current State	Lo que realmente existe, registrado en `terraform.tfstate`.
Plan (DAG)	El grafo que calcula qué cambiar y en qué orden.
Apply	Las llamadas reales al provider para alcanzar el estado deseado.

> **¿Por qué empezamos con el provider local?**
>
> Crea archivos en tu propia computadora: no necesita cuenta de nube ni cuesta dinero. **El flujo es exactamente el mismo que con AWS**; si lo entiendes con archivos, lo entiendes con servidores.

5. Tu primer proyecto: main.tf

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.4"
    }
  }
}

resource "local_file" "hola_terraform" {
  content  = "Hola desde Terraform!"
  filename = "${path.module}/output/hola.txt"
}
```

Elemento	Qué significa
`terraform {}`	Bloque de configuración global del proyecto.
`required_providers`	Declara qué providers necesita el proyecto.
`source`	De dónde se descarga el provider (Terraform Registry).

Elemento	Qué significa
`version = "~> 2.4"`	Acepta cualquier 2.x desde la 2.4 (2.5, 2.9…), pero nunca la 3.0.
`local_file`	**Tipo** de recurso; lo define el provider.
`hola_terraform`	**Nombre lógico**; solo existe dentro de Terraform. La dirección completa es `local_file.hola_terraform`.
`content` / `filename`	Contenido del archivo y ruta donde se creará. `path.module` es la carpeta del .tf.

6. El flujo: init → plan → apply → destroy

Comando	Qué hace
`terraform init`	Descarga los providers. Crea `.terraform/` (el binario, **no** va a Git) y `.terraform.lock.hcl` (la versión exacta, **sí** va a Git). Se repite al agregar un provider nuevo.
`terraform plan`	Muestra lo que va a cambiar **sin tocar nada**. Algunos valores aparecen como `(known after apply)` porque solo se conocen al crear el recurso.
`terraform apply`	Muestra el plan, pide confirmación (`yes`), crea los recursos y actualiza el tfstate. Con `-auto-approve` no pregunta.
`terraform destroy`	Elimina todos los recursos que Terraform administra en ese proyecto.

Cómo leer los símbolos del plan: `+` crear · `~` modificar en sitio · `-/+` destruir y volver a crear · `-` destruir.

Lo que comprobamos en vivo:

- **Idempotencia:** correr `terraform apply` otra vez responde `No changes`. Si la realidad coincide con el código, Terraform no hace nada.
- **Drift:** al editar `hola.txt` a mano, el siguiente `plan` lo detecta y propone volver a crearlo. Lo que se cambia a mano, se pierde: el código es la fuente de verdad.
- **Reemplazo:** al cambiar `content` en el código, el plan muestra `-/+` y `# forces replacement`. Con un archivo no importa; con una base de datos de producción, sí. **Siempre lean el plan.**

7. terraform.tfstate: la memoria de Terraform

- Es un archivo **JSON** con el estado de **todos** los recursos que Terraform administra (tipo, nombre, atributos).
- Para consultarlo sin abrir el JSON: `terraform state list` (lista los recursos) y `terraform show` (estado formateado).
- **Nunca se edita a mano** y **nunca se sube a Git**: puede contener datos sensibles como contraseñas.
- En equipos se guarda en un **backend remoto** (S3, Terraform Cloud) para que todos compartan el mismo estado.

8. Ejercicio estrella: amplía el proyecto

Agrega estos dos recursos a tu `main.tf`:

```
resource "local_file" "config" {
  content  = "ambiente=dev\nversion=1.0\n"
  filename = "${path.module}/output/config.env"
}

resource "local_file" "script" {
  content         = "#!/bin/bash\necho \"Terraform!\"\n"
  filename        = "${path.module}/output/run.sh"
  file_permission = "0755"   # en Windows se ignora
}
```

Pasos: 1) `terraform plan` y busca los `+` nuevos · 2) `terraform apply` y verifica los archivos en `output/` · 3) revisa el tfstate: ¿cuántos recursos hay? · 4) edita `config.env` a mano y observa qué propone `terraform plan` · 5) `terraform destroy` para limpiar.

9. Los 5 comandos que ya conoces

Comando	Para qué sirve
`terraform init`	Inicializa el proyecto y descarga los providers.
`terraform plan`	Muestra qué va a cambiar, sin ejecutar nada.
`terraform apply`	Aplica los cambios (pide confirmación).
`terraform destroy`	Elimina todos los recursos administrados.
`terraform show`	Muestra el estado actual formateado.

> ***"Si solo aprenden sintaxis de Terraform, el curso falla. Si aprenden a pensar infraestructura como código, pueden automatizar cualquier nube."***